

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3

Выполнил:

Смирнов Александр

Группа J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г

Задача

В рамках третьей лабораторной работы необходимо было осуществить миграцию клиентской части платформы из многостраничного приложения (MPA) на ванильном JavaScript в одностраничное приложение (SPA) с использованием фреймворка Vue 3 и сборщика Vite.

Реализованный функционал:

- Перенос интерфейса на Vue 3 (Composition API) и настройка маршрутизации через vue-router.
- Разделение архитектуры на изолированные слои: UI-компоненты (Views/Components), управление состоянием (Composables) и сетевое взаимодействие (API).
- Настройка централизованного HTTP-клиента (Axios) с обработкой таймаутов (8000 мс) и нормализацией ошибок.
- Улучшение доступности (a11y) и пользовательского опыта (UX): замена эмодзи на инлайн SVG-иконки, добавление aria-атрибутов интерактивным элементам.
- Повышение надежности клиентской логики: генерация уникальных идентификаторов (crypto.randomUUID) и нормализация ввода.
- Реализация демонстрационной безопасности: клиентское хеширование паролей (SHA-256) и проверка их надежности.
- Создание рекурсивного компонента для системы вложенных комментариев.

Ход работы

1. Инициализация проекта и настройка маршрутизации

Проект был инициализирован с использованием сборщика Vite. Настроена маршрутизация с помощью vue-router (файл index.js). Добавлены навигационные guard'ы (meta.requiresAuth) для защиты приватных маршрутов (создание карточек, профиль) и редиректа неавторизованных пользователей. Создана базовая разметка с помощью компонента BaseLayout.vue, включающего глобальные компоненты AppHeader.vue и AppFooter.vue.

2. Организация API-слоя и сетевых запросов

Вместо разрозненных Fetch-запросов был внедрен централизованный API-слой. В файле http.js настроен экземпляр Axios с базовым URL и интерсептором для перехвата и нормализации ошибок (в т.ч. серверных 500 и таймаутов).

Взаимодействие с mock-бэкендом (json-server) разделено на независимые модули: items.js, comments.js, notifications.js и users.js. Каждое API-обращение обернуто в функцию с понятной сигнатурой.

3. Управление состоянием через Composables

Логика приложения переведена на Composition API. Для инкапсуляции реактивной логики и side-эффектов были разработаны переиспользуемые функции - Composables. Например, useAuth.js отвечает за сессию пользователя и взаимодействие с localStorage, а useTheme.js управляет переключением светлой и темной тем.

4. Рекурсивные компоненты и улучшение UX/a11y

Система обсуждений была переписана с использованием рекурсивного компонента CommentThread.vue. Комментарии динамически фильтруются по parentId, а рекурсивный вызов компонента внутри самого себя позволил элегантно отобразить дерево ответов любой вложенности без дублирования кода. Для улучшения доступности (a11y) старые текстовые эмодзи заменены на SVG-графику (с управлением цветом черезcurrentColor) и добавлены aria-label для кнопок.

5. Клиентская безопасность и генерация данных

В рамках демонстрации внедрена функция hashPassword (утилита security.js), использующая алгоритм SHA-256 для хеширования пароля перед отправкой (для mock-бэкенда), а также валидатор isStrongPassword. Для генерации надежных идентификаторов новых сущностей задействован метод crypto.randomUUID() с fallback-решением (id.js).

Вывод

В результате выполнения лабораторной работы проект был успешно переведен на современную компонентную SPA-архитектуру на базе Vue 3 и Vite.

Были получены и закреплены следующие навыки:

- Проектирование и разработка Single Page Application с использованием Composition API.
- Настройка клиентской маршрутизации и защиты приватных путей (vue-router).

- Разделение монолитного кода на изолированные слои: UI, Composables, API-модули.
- Использование Axios для создания надежного HTTP-клиента с интерсепторами.
- Разработка рекурсивных компонентов во Vue для рендеринга древовидных структур данных.
- Применение современных практик веб-доступности (a11y) и работа с inline SVG.
- Базовое понимание клиентской криптографии (SHA-256, UUID) для демонстрации концепций безопасности.

Проект стал масштабируемым, поддерживаемым и устойчивым веб-приложением с плавным пользовательским интерфейсом, полностью готовым к интеграции с реальным бэкендом (замене json-server).